

DSA Lab Challenge: Micromouse-Style Maze Solver

Objective

In this challenge, each team must implement a maze-solving algorithm for a Micromouse-style simulator.

The mouse must:

1. explore an unknown maze,
2. find the blue goal room during the `SEARCH_RUN`,
3. keep the knowledge learned during exploration,
4. restart from the start point,
5. reach the goal room again during the `SPEED_RUN`.

The goal is not only to finish, but to finish efficiently.

What You Receive

You are given:

- a compiled simulator framework,
- a starter solver file,
- a GUI to test your algorithm,
- example mazes for understanding the problem.

You are **not** given:

- the hidden maze generator,
- the hidden evaluation maps,
- the teacher evaluation logic.

What You Must Implement

You must implement your algorithm inside:

`src/student/StudentSolverImpl.java`

Your solver should decide how the mouse:

- explores the maze,
- detects dead ends,
- backtracks,
- stores discovered information,
- chooses directions,
- controls speed,
- improves the second run after learning the maze.

Allowed Algorithm Ideas

You may use any suitable algorithm, including:

- DFS,
- BFS,
- recursive backtracking,
- flood fill,
- shortest path algorithms,
- heuristic search,
- hybrid strategies.

You are encouraged to design your own approach.

Rules

- The maze is initially unknown to your solver.
- The solver only receives local perception from the API.
- The blue cells form the goal room.
- The first phase is the `SEARCH_RUN`.
- When the mouse reaches the goal room for the first time, the search run ends.
- The simulator then resets the mouse to the start for the `SPEED_RUN`.

- The solver keeps the knowledge it learned during the search run.
- The run finishes when the mouse reaches the goal room again during the speed run.
- There is no direct **BACKWARD** action.
- To go back, the mouse must turn around and move forward.
- Hard-coding one path is not acceptable.
- Your solver must work on many different mazes.

Deliverables

Each team must submit:

- `StudentSolverImpl.java`

Optional:

- a short explanation of the algorithm,
- important design decisions,
- any known limitations.

How To Run The Student Kit

Inside the student kit folder, students can use:

- `./compile.sh` to compile `StudentSolverImpl.java`
- `./run_gui.sh` to compile and launch the simulator GUI

Typical usage:

```
cd dist/student-kit
./compile.sh
./run_gui.sh
```

If the operating system reports a permission error, students may first run:

```
chmod +x compile.sh run_gui.sh
```

Minimum Correctness Requirement

A valid solution should:

- compile successfully,
- run in the provided simulator,
- complete the search run on non-trivial mazes,
- attempt a meaningful speed run using discovered information.

Evaluation

Your solution will be evaluated using hidden maps.

Evaluation may include:

- completion of the search run,
- completion of the speed run,
- total time,
- search run time,
- speed run time,
- number of actions,
- path efficiency,
- collision count,
- speed control quality.

Competition and Winner

To determine the winner, each solution will be tested on at least:

- 5 hidden maps, and possibly
- 10 hidden maps.

These maps will include:

- difficult mazes,
- extremely hard “hell level” mazes.

The winner will be the team whose solver performs best overall on the hidden evaluation set, not only on the example mazes provided in class.

Important Advice

- Do not optimize for only one map.
- Make your solver robust.
- Make your solver able to recover from dead ends.
- Make your solver remember useful information between the two runs.
- Think about both correctness and performance.

Academic Integrity

- You may discuss high-level ideas if your instructor allows it.
- Your team must write its own final implementation.
- Copying another team's solver is not allowed.